SQL Project : Student Grade Tracker

Built a student grade tracking system using SQLite with 3 related tables: students, courses, and grades. Inserted data for 8 students across 6 courses and wrote queries to analyze academic performance.

Queries written:
- Joined all three tables to get a full readable view of student grades
- Calculated average grade per student using AVG() and GROUP BY
- Sorted students by average grade using ORDER BY
- Labeled grades as A, B, C, D, F using CASE
- Used HAVING to filter students above a grade threshold

Key concepts practiced: JOIN across multiple tables, GROUP BY, aggregate functions, CASE statements, and HAVING vs WHERE.

Tools: SQLite, DB Browser for SQLite

---

1. Creating a table of students, courses, and grades:

```
CREATE TABLE students(
        id INTEGER,
        name TEXT,
        major TEXT
);

CREATE TABLE courses(
        id INTEGER,
        course_name TEXT,
        credits INTEGER
);
```

```
CREATE TABLE grades(
        student_id INTEGER,
        course_id INTEGER,
        grade REAL
);
```

2. Adding values into the tables:

```
-- Students
INSERT INTO students VALUES (1, 'Alice Johnson', 'Computer Science');
INSERT INTO students VALUES (2, 'Brian Lee', 'Mathematics');
INSERT INTO students VALUES (3, 'Carmen Rivera', 'Computer Science');
INSERT INTO students VALUES (4, 'David Kim', 'Physics');
INSERT INTO students VALUES (5, 'Emma Patel', 'Computer Science');
INSERT INTO students VALUES (6, 'Felix Nguyen', 'Mathematics');
INSERT INTO students VALUES (7, 'Grace Chen', 'Physics');
INSERT INTO students VALUES (8, 'Hassan Ali', 'Computer Science');

-- Courses
INSERT INTO courses VALUES (1, 'Intro to Programming', 3);
INSERT INTO courses VALUES (2, 'Data Structures', 3);
INSERT INTO courses VALUES (3, 'Calculus I', 4);
INSERT INTO courses VALUES (4, 'Linear Algebra', 3);
INSERT INTO courses VALUES (5, 'Physics I', 4);
INSERT INTO courses VALUES (6, 'Database Systems', 3);

-- Grades (grade out of 100)
INSERT INTO grades VALUES (1, 1, 95);
INSERT INTO grades VALUES (1, 2, 88);
INSERT INTO grades VALUES (1, 6, 92);
INSERT INTO grades VALUES (2, 3, 78);
INSERT INTO grades VALUES (2, 4, 85);
INSERT INTO grades VALUES (3, 1, 90);
```

INSERT INTO grades VALUES (3, 2, 76);
INSERT INTO grades VALUES (3, 6, 88);
INSERT INTO grades VALUES (4, 3, 95);
INSERT INTO grades VALUES (4, 5, 89);
INSERT INTO grades VALUES (5, 1, 72);
INSERT INTO grades VALUES (5, 2, 68);
INSERT INTO grades VALUES (5, 6, 75);
INSERT INTO grades VALUES (6, 3, 91);
INSERT INTO grades VALUES (6, 4, 87);
INSERT INTO grades VALUES (7, 3, 83);
INSERT INTO grades VALUES (7, 5, 79);
INSERT INTO grades VALUES (8, 1, 98);
INSERT INTO grades VALUES (8, 2, 94);
INSERT INTO grades VALUES (8, 6, 96);

3. Checking to see if the values are inserted into the table:

SELECT * FROM students;

id	name	major
1	Alice Johnson	Computer Science
2	Brian Lee	Mathematics
3	Carmen Rivera	Computer Science
4	David Kim	Physics
5	Emma Patel	Computer Science
6	Felix Nguyen	Mathematics
7	Grace Chen	Physics
8	Hassan Ali	Computer Science

SELECT * FROM grades;

	student_id	course_id	grade
1	1	1	95
2	1	2	88
3	1	6	92
4	2	3	78
5	2	4	85
6	3	1	90
7	3	2	76
8	3	6	88
9	4	3	95
10	4	5	89
11	5	1	72
12	5	2	68
13	5	6	75
14	6	3	91
15	6	4	87
16	7	3	83
17	7	5	79
18	8	1	98
19	8	2	94
20	8	6	96

SELECT * FROM courses;

	id	course_name	credits
1	1	Intro to Programming	3
2	2	Data Structures	3
3	3	Calculus I	4
4	4	Linear Algebra	3
5	5	Physics I	4
6	6	Database Systems	3

4. Query to calculate GPA per student using AVG():

```
SELECT students.name, AVG(grades.grade) AS average_grade
FROM grades
JOIN students ON grades.student_id = students.id
GROUP BY students.name;
```

	name	average_grade
1	Alice Johnson	91.6666666666667
2	Brian Lee	81.5
3	Carmen Rivera	84.6666666666667
4	David Kim	92.0
5	Emma Patel	71.6666666666667
6	Felix Nguyen	89.0
7	Grace Chen	81.0
8	Hassan Ali	96.0

5. Sort students by average grade highest to lowest:

```
SELECT students.name, AVG(grades.grade) AS average_grade
FROM grades
JOIN students ON grades.student_id = students.id
GROUP BY students.name
ORDER BY average_grade DESC;
```

	name	average_grade
1	Hassan Ali	96.0
2	David Kim	92.0
3	Alice Johnson	91.666666666666
4	Felix Nguyen	89.0
5	Carmen Rivera	84.666666666666
6	Brian Lee	81.5
7	Grace Chen	81.0
8	Emma Patel	71.666666666666

6. Full readable table combining students, grades, courses:

```
SELECT students.name, students.major, courses.course_name,
courses.credits, grades.grade
FROM grades
JOIN students ON grades.student_id = students.id
JOIN courses ON grades.course_id = courses.id;
```

	name	major	course_name	credits	grade
1	Alice Johnson	Computer Science	Intro to Programming	3	95
2	Alice Johnson	Computer Science	Data Structures	3	88
3	Alice Johnson	Computer Science	Database Systems	3	92
4	Brian Lee	Mathematics	Calculus I	4	78
5	Brian Lee	Mathematics	Linear Algebra	3	85
6	Carmen Rivera	Computer Science	Intro to Programming	3	90
7	Carmen Rivera	Computer Science	Data Structures	3	76
8	Carmen Rivera	Computer Science	Database Systems	3	88
9	David Kim	Physics	Calculus I	4	95
10	David Kim	Physics	Physics I	4	89
11	Emma Patel	Computer Science	Intro to Programming	3	72
12	Emma Patel	Computer Science	Data Structures	3	68
13	Emma Patel	Computer Science	Database Systems	3	75
14	Felix Nguyen	Mathematics	Calculus I	4	91
15	Felix Nguyen	Mathematics	Linear Algebra	3	87
16	Grace Chen	Physics	Calculus I	4	83
17	Grace Chen	Physics	Physics I	4	79
18	Hassan Ali	Computer Science	Intro to Programming	3	98
19	Hassan Ali	Computer Science	Data Structures	3	94
20	Hassan Ali	Computer Science	Database Systems	3	96

7. Label grades as A, B, C, D, F using CASE:

```
SELECT students.name, courses.course_name, grades.grade,
CASE
   WHEN grades.grade >= 90 THEN 'A'
   WHEN grades.grade >= 80 THEN 'B'
   WHEN grades.grade >= 70 THEN 'C'
   WHEN grades.grade >= 60 THEN 'D'
   ELSE 'F'
END AS letter_grade
FROM grades
JOIN students ON grades.student_id = students.id
JOIN courses ON grades.course_id = courses.id;
```

	name	course_name	grade	letter_grade
1	Alice Johnson	Intro to Programming	95	A
2	Alice Johnson	Data Structures	88	B
3	Alice Johnson	Database Systems	92	A
4	Brian Lee	Calculus I	78	C
5	Brian Lee	Linear Algebra	85	B
6	Carmen Rivera	Intro to Programming	90	A
7	Carmen Rivera	Data Structures	76	C
8	Carmen Rivera	Database Systems	88	B
9	David Kim	Calculus I	95	A
10	David Kim	Physics I	89	B
11	Emma Patel	Intro to Programming	72	C
12	Emma Patel	Data Structures	68	D
13	Emma Patel	Database Systems	75	C
14	Felix Nguyen	Calculus I	91	A
15	Felix Nguyen	Linear Algebra	87	B
16	Grace Chen	Calculus I	83	B
17	Grace Chen	Physics I	79	C
18	Hassan Ali	Intro to Programming	98	A
19	Hassan Ali	Data Structures	94	A
20	Hassan Ali	Database Systems	96	A